Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr

CONTENT

3.1. Abstract Data Types (ADTs)

- One of the basic rules concerning programming is that no routine should ever exceed a page.
- This is accomplished by breaking the program down into *modules*.
- Each module is a logical unit and does a specific job.
- A module's size is kept small by calling other modules.
- Modularity has several advantages:
  - It is much easier to debug small routines than large routines.
  - it is easier for several people to work on a modular program simultaneously.
  - A well-written modular program places certain dependencies in only one routine, making changes easier.

3.1. Abstract Data Types (ADTs)

- An *abstract data type* (ADT) is a set of operations.
- Abstract data types are mathematical abstractions; nowhere in an ADT's definition is there any mention of *how* the set of operations is implemented. This can be viewed as an extension of modular design.
- Objects such as lists, sets, and graphs, along with their operations, can be viewed as abstract data types, just as integers, reals, and booleans are data types, and also we can define our own data type by the use of 'struct' component in C.

3.1. Abstract Data Types (ADTs)

- For the set ADT, we might have such operations as *union*, *intersection*, *size*, and *complement*.
  - Alternately, we might only want the two operations *union* and *find*, which would define a different ADT on the set.
- The basic idea is that the implementation of these operations is written once in the program, and any other part of the program that needs to perform an operation on the ADT can do so by calling the appropriate function.

3.2. The List ADT

- We will deal with a general list of the form $a_1, a_2, a_3, \ldots, a_n$.
- *We say that the size of this list is n.*
- *We will call the special list of size 0 as a null list.*
- For any list except the null list, we say that a_{i+1} *follows* a_i $(i < n)$ *and that* a_{i-1} *precedes* a_i $(i > 1)$.
  - *The first element of the list is* a_1.
  - *the last element is* a_n.
  - *We will not define the predecessor of* a_1 *or the successor of* a_n.
  - *The position of element* a_i *in a list is i.*
- *Throughout this discussion, we will assume, to simplify matters, that the elements in the list are integers, but in general, arbitrarily complex elements are allowed.*

3.2. The List ADT

- Some popular operations are *print_list* and *make_null*, which do
  - *Find*
  - *Insert*
  - *Delete*
  - *Find_kth*
  - ...

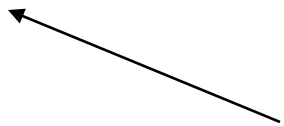
3.2.1. Simple Array Implementation of Lists

- Arrays require that all elements must be of the same data type.
- Many times it is necessary to use and group information of different data types.
  - An example could be a simple address book which usually includes a name for each entry , an email address, and phone number.
- C supports data structures that can store combinations of character, integer floating point and enumerated type data. They are called a *structs*.

The Struct Statement

Here is an example struct statement.

```
#include <stdio.h>
struct line {
    int x1, y1;  /* coordinates of a point lying on the line*/
    int x2, y2;  /* coordinates of a point lying on the line */
    };
main()
{
struct line line1;
.
.
}
```



This defines the variable line1 to be a variable of type line

Typedef & Struct

- Generally, *typedef* is used in combination with *struct* to declare a synonym for a structure:

```
typedef struct example                    /* Define a structure
*/
{
   int label ;
   char letter;
   char fname[20] ;
} MyStructure ;              /* The "alias" is MyStructure */

MyStructure  MyStruct ;    /* Create a struct variable */
```

Typedef

- Typedef allows us to associate a name with a structure (or other data type).
- Put typedef at the start of your program.

```
typedef struct line {
    int x1, y1;
    int x2, y2;
    } LINE;

int main()
{
LINE line1;

}
```

line1 is now a structure of line type

Accessing Struct Members

- Individual members of a *struct* variable may be accessed using the structure member operator (the dot, "."):

```
MyStruct.letter ;
```

or , if a pointer to the *struct* has been declared and initialized

```
MyStructure   *myptr = &MyStruct ;
```

by using the structure pointer operator (the "->"):

```
myptr -> letter ;
```

which could also be written as:

```
(*myptr).letter ;
```

Accessing Struct Members

For example :

```
LINE line1;

line1.x1= 3;
line1.y1= 5;
line1.x2= 7;

if (line1.y2 == 3)
        printf ("Y co-ordinate of end is 3\n");
```

What Else Can We Do With Structs?

- We can pass and return structs from functions.
  - Note that the function prototype can be declared based on the declared-struct.

For example:

```
typedef struct line {
    int x1,y1;
    int x2,y2;
} LINE;

/* Function takes a line and returns a perpendicular line */
LINE find_perp_line (LINE);
```

Example: Complex numbers

```
typedef struct complex {   float imag;
                           float real;
                         } CPLX;

CPLX mult_complex (CPLX, CPLX);

int main()
{
/* Main goes here */
}

CPLX mult_complex (CPLX no1, CPLX no2)
{
   CPLX answer;
   answer.real= no1.real*no2.real - no1.imag*no2.imag;
   answer.imag= no1.imag*no2.real + no1.real*no2.imag;
   return answer;
}
```

How Structs Can Contain Themselves

- A struct cannot literally contain itself.

```
struct silly_struct { /* This doesn't work */
    struct silly_struct s1;
};
```

- But it can contain a pointer to the same type of struct.

```
struct good_struct { /* This does work */
    struct *good_struct s2;
};
```

Disadvantages of Arrays

- The size of the array is fixed.
  - In case of **dynamically resizing** the array from size S to 2S, we need 3S units of available memory.
  - Programmers allocate arrays which seem **"large enough"**. This strategy has two disadvantages:

    (a) Most of the time there are just 20% or 30% elements in the array and 70% of the space in the array really is wasted.

    (b) If the program ever needs to process more than the declared size, the code breaks.

Disadvantages of Arrays

- Elements of the array are stored contiguously.
- Inserting (and deleting) elements into the middle of the array is potentially expensive because existing elements need to be shifted over to make a room.

Questions and Answers

Thank you ...

Prof. Dr. Ayla ŞAYLI

http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr